ProbGen: A Browser-Based Randomised Practice Problem Generator

Mostafa Shehata,
Friedrich-Alexander-Universität Erlangen-Nürnberg

Abstract

Creating sufficient numbers of diverse and well-structured exercises is a practical challenge in computer science education, particularly for topics involving mathematical and algorithmic reasoning. This project develops ProbGen, an interactive browser-based environment for generating and practising problems in scalable multiple problems. ProbGen builds on an existing Scala-based problem-generation framework and extends it with a structured expression and document-rendering pipeline for web use. Problem instances are generated from randomized parameters and represented using a common problem and subproblem model. Mathematical expressions are represented explicitly rather than as plain strings, enabling consistent generation, STeX output, and HTML rendering. The application is compiled to JavaScript using ScalaJS, so problem generation, rendering, and answer checking can be performed directly in the browser without requiring a dedicated application server. Each subproblem provides an answer field and immediate feedback, while the checking mechanism can be specialized for different problem domains. The resulting system provides a reusable foundation for interactive practice and can be extended with additional problem generators while retaining the existing rendering and interaction infrastructure.

Index Terms

automated problem generation, interactive learning, formative assessment, ScalaJS, online-exercises, STeX, HTML, CSS, web application

I. Introduction

The project began as a Scala tool that generated LaTeX source for exam problems by sampling random parameters, solving the resulting problem internally, and printing the question and solution to the terminal. While this approach produced structurally correct problems, it offered no way for students to interact with them: there was no input field, no answer checking, and no feedback.

The work described in this paper transforms ProbGen into a fully browser-native interactive application. The transformation proceeds in three directions.

First, the entire Scala codebase is compiled to JavaScript using Scala.js, the resulting application runs as two static files `index.html` and `main.js` that can be opened directly in any browser without installation.

Second, a complete HTML rendering pipeline is implemented for the mathematical expression AST and the STeX document model (`stex.scala`). Every node gains a `toHTML` method that produces styled browser output and a `toText` method that produces clean plain text for answer comparison. Mathematics is rendered using styled HTML `<span>` elements rather than any external library, avoiding dependencies on MathJax or KaTeX and working correctly with the custom STeX macros the generators produce.

Third, a structured answer-checking framework is introduced. Every subproblem declares a `checkSolution` method that returns one of three outcomes: `Correct()`, `Incorrect(hint)`, or `NotCheckable(expected)`. For search problems, checking is fully semantic the submitted action sequence is executed against the transition model and any path reaching a goal state is accepted, not only the specific sequence stored in the model answer.

The remainder of this paper is organised as follows.

Section II describes the original architecture and what it lacked. Section III covers the complete implementation in the following parts: overall system architecture, expression representation and rendering, the STeX rendering pipeline, interactive answer checking, browser integration, random problem generation, and build configuration. Section IV presents the user interface and sample outputs from all three problem domains. Section V evaluates correctness, answer-checking quality, problem diversity, and performance. Section VI concludes with a discussion of limitations and planned future work.

The full source code and build instructions are available at https://github.com/UniFormal/ProbGen. Also the deployed page available at https://uniformal.github.io/ProbGen/.

II. System Architecture

A. Original System Architecture

Before the modifications presented in this work, ProbGen was primarily designed as a JVM-based problem-generation system. The core functionality was implemented in Scala and focused on generating structured problem instances and exporting them into a LaTeX/STeX-based representation. The architecture separated problem generation from the document representation, with problem generators creating domain-specific problems and the STeX layer responsible for representing and formatting the generated content.

The original system was therefore mainly a *generation and document production pipeline*, rather than a fully interactive browser application. The generated STeX documents could represent mathematical expressions, problem statements, tables, and subproblems, but the original rendering layer did not provide the complete browser-based interaction implemented in the updated system. In particular, answer fields, client-side answer checking, and dynamic regeneration of problems were not integrated into a single browser workflow.

The main components of the original architecture can be summarised as:

- **Problem generators:** created domain-specific problem instances using Scala and the shared problem abstractions.
- **Expression model:** represented mathematical terms, formulas, operators, sets, tuples, and other structured expressions.
- **STeX document model:** represented the generated problem statements and their mathematical content in a structured document format.
- **LaTeX/STeX output:** provided the primary representation for producing formatted mathematical documents.
- **Problem and subproblem abstractions:** organised each generated exercise into a collection of subproblems with questions and solutions.

The main objective of the subsequent development was therefore not to replace the existing generation framework, but to extend it with a browser execution and interaction layer. The updated architecture retains the problem-generation and expression-model components while adding Scala.js compilation, HTML rendering, interactive answer checking, dynamic problem regeneration, and static web deployment.

B. Problem Domains

ProbGen covers five problem domains corresponding to major topics:

- **MDP Problems.**
- **Probability Problems.**
- **Search Problems.**
- **CSP Problems.**
- **Adversial Search Problems.**

C. New Architecture Core Layers

The system is structured in four layers, shown in Figure. 1. **Layer 1 Generators.** Each domain has a generator object that randomly instantiates a problem, solves it internally, and rejects instances that are trivial or degenerate.

Layer 2 Expression AST (`expression.scala`). A typed expression language shared by all generators. Nodes include `Var` (variable reference), `Lit` (literal value), `Apply` (function application), `Pred` (predicate), `BigApply` (binding operators such as $\sum$, max), and `Prob` (probability notation). Each node originally had only a `toSTeX` method producing LaTeX strings.

Layer 3 STeX document model (`stex.scala`). Wraps expression nodes in document structure: `SDocument`, `SFragment`, `SProblem`, `SSubproblem`, `SSolution`, `STabular`, and list/text nodes. The `x"..."` string interpolator converts Scala objects into the appropriate STeX nodes.

Layer 4 Problem logic (`problem.scala`). The `Problem` trait hosts inner `Subproblem` objects, each with `question()` and `solution()` methods returning `SText` nodes. `GroupConstraint` declarations control which subsets of subproblems are chosen for a given instance.

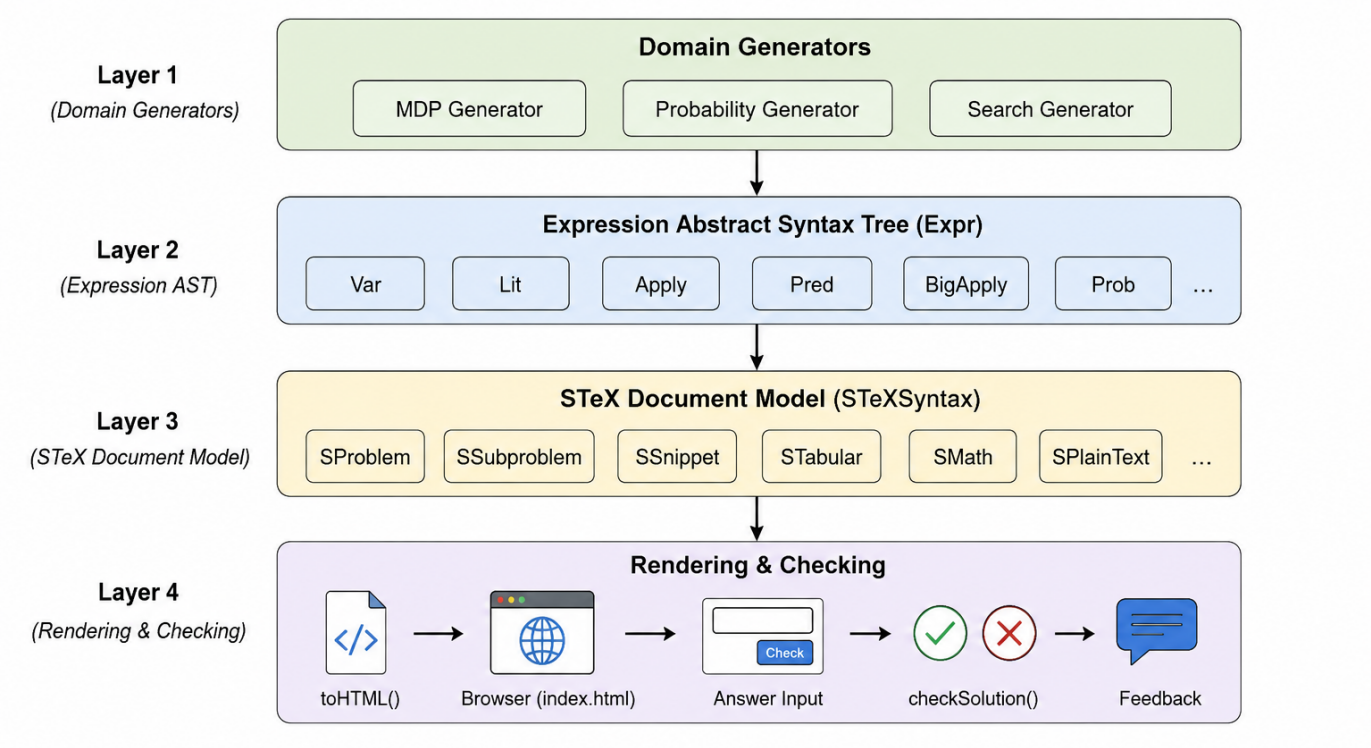


Fig. 1: ProbGen system architecture.

III. Implementation

A. System Architecture

The primary objective of this project was to extend the existing ProbGen framework from a static STeX generator into an interactive browser-based educational application while preserving the framework's original architecture. Rather than modifying the individual problem generators to produce HTML directly, the implementation introduces additional rendering and interaction layers that transform the existing document representation into browser-compatible output. This approach preserves the modularity of the original system and ensures that problem generation remains independent of presentation.

Figure 1 illustrates the overall architecture of the implemented system. Starting from a user request to generate a new exercise sheet, the framework executes a sequence of processing stages that culminates in an interactive HTML document displayed within the browser.

The complete workflow consists of the following stages:

1) Domain-specific generators construct randomized exercises.
2) Generated problems are represented as structured STeX document objects.
3) Mathematical expressions are stored as abstract syntax trees (ASTs).
4) The rendering layer traverses the document and converts every syntax object into HTML.
5) The generated HTML is inserted into the browser.
6) User answers are evaluated through the interactive answer-checking framework.

The architecture follows a layered design in which each layer has a clearly defined responsibility. Problem generators are responsible only for constructing semantic representations of exercises and are completely unaware of the eventual output format. Likewise, the rendering layer performs only presentation-related tasks and does not participate in problem generation or answer evaluation. This separation considerably improves maintainability and allows individual components to evolve independently.

At the core of the framework are two complementary hierarchies, the first is the expression hierarchy centred around the abstract class `Expr`, which represents every mathematical expression appearing in generated problems. Expressions are constructed by the generators and later rendered into STeX, HTML, or normalized plain text depending on the application.

The second is the document hierarchy rooted at `STeXSyntax`. This hierarchy models the logical structure of an exercise sheet independently of its presentation. Documents, problems, subproblems, tables, lists, mathematical snippets, and solutions are all represented using subclasses of this common abstraction.

One of the main architectural goals of the project was to preserve these semantic representations and extend only the rendering stage. Consequently, the existing generators required only minimal modifications despite the addition of an entirely new browser interface.

The browser interface itself is implemented using Scala.js. Instead of rewriting the application in JavaScript, the existing Scala implementation is compiled directly into JavaScript, allowing all rendering logic, answer checking, and user interaction to remain within the same programming language. Communication between the generated HTML and the Scala implementation is achieved through exported JavaScript functions, allowing browser events to invoke Scala methods directly.

Overall, the resulting architecture preserves the modular organization of the original ProbGen framework while extending it with browser rendering and interactive functionality. By separating problem generation, document representation, rendering, and user interaction into independent layers, the framework remains extensible and readily supports additional output formats or future problem generators.

B. Expression Representation:

The Expr AST:

The `Expr` abstract class is the root of the expression AST. Every mathematical object in a generated problem is an `Expr` node. The hierarchy has two branches:

- **`Term`** integer- or string-valued expressions: `Var` (named variable), `Lit` (literal value with a domain tag), `Apply` (n-ary function application), `BigApply` (binding operators such as $\sum$ and max), and `Prob` (probability notation $P(\cdot)$ and $P(\cdot|\cdot)$).
- **`Form`** Boolean-valued formulas: `Pred` (predicate application, *e.g.* $s \in S$), `Conn` (logical connective), and `BVar` (Boolean variable).

Before this work, the abstract class declared only `toSTeX`. The implementation adds `toHTML` and `toText` as abstract methods, forcing every concrete node to provide all three renderings:

```
sealed abstract class Expr {
  def unary_~ = SMath(this)   // convenience: ~expr wraps in SMath

  def toSTeX: String   // original: LaTeX output for .tex files
  def toHTML: String   // new: HTML fragment for browser rendering
  def toText: String   // new: plain text for answer comparison
}
```

Listing 1: Abstract Expr class with the three rendering methods.

The MathHTML Token Converter:

The `MathHTML` object converts individual LaTeX tokens found inside `$...$` segments in the `x"..."` interpolator into their HTML equivalents:

```
object MathHTML {
  // Convert a single LaTeX token to an HTML fragment
  def token(s: String): String = s.trim match {
    case "\\gamma"  => "<i>\u03B3</i>"       //
    case "\\pi"     => "<i>\u03C0</i>"       //
    case "\\to"     => "<span class='sym'>\u2192</span>"  //
    case "\\times"  => "<span class='sym'>\u00D7</span>"  //
    case "\\cdot"   => "<span class='sym'>\u22C5</span>"  //
    case "\\infty"  => "<span class='sym'>\u221E</span>"  //
    case "\\hline"  => ""
```

```
    case cmd if cmd.startsWith("\\mathtt{") && cmd.endsWith("}") =>
      val inner = cmd.stripPrefix("\\mathtt{").stripSuffix("}")
      s"<span class='mathtt'>$inner</span>"
    case cmd if cmd.startsWith("\\mathrm{") && cmd.endsWith("}") =>
      cmd.stripPrefix("\\mathrm{").stripSuffix("}")
    case other => other  // pass through unknown tokens unchanged
  }

  // Handle compound expressions such as "4 \times 3"
  // by splitting on whitespace and mapping each token individually
  def apply(s: String): String = {
    val parts = s.trim.split("\\s+")
    if (parts.length <= 1) token(s.trim)
    else parts.map(p => token(p.trim)).mkString("␣")
  }
}
```

Listing 2: MathHTML object: token-level LaTeX to HTML conversion.

The compound-expression handling is essential for cases such as `$$4 \times 3$$` in the grid MDP description, where the `x"..."` interpolator emits a `SPlainText` node with body `"$4 \times 3$"` that must be split and mapped before wrapping in a math span.

The x-Interpolator and the STeXSyntax Passthrough Fix:

The `x"..."` string interpolator is defined in the `SText` companion object. It converts each Scala value passed as a `${...}` argument into the appropriate STeX node. A critical bug existed in the original fallback: when a value such as `SMath(RangeSet(a,l))` was passed as an argument, the match fell through to `SPlainText(a.toString)`, producing the raw LaTeX string `"$\range{a}{l}$"` as plain text in the output.

The fix adds a `STeXSyntax` guard as the first and highest-priority arm of the match:

```
def x(args: Any*): SText = {
  // ... build partsS from StringContext.parts ...
  args.toList.foreach { arg =>
    val argS: STeXSyntax = arg match {
      case t: STeXSyntax => t          // FIXED: pass through unchanged
      case s: String     => SPlainText(s)
      case f: Form       => SMath(f)
      case a => Expr.fromAnyO(a) match {
        case Some(e) => SMath(e)
        case None    => SPlainText(a.toString)
      }
    }
    snippets ::= argS
    snippets ::= partsS.head
    partsS = partsS.tail
  }
  SSnippet(snippets.reverse)
}
```

Listing 3: Fixed x-interpolator: STeXSyntax passthrough.

With this fix, passing `eventNamesShort` (which is of type `SMath`) to `x"...in terms of the ${eventNamesShort}..."` correctly preserves the `SMath` node, allowing it to render as `<span class="math">{a, ..., l}</span>` in the browser.

C. Interactive Answer Checking

The CheckResult Algebraic Type:

Answer checking is structured around a sealed algebraic type with three distinct outcomes, following the supervisor's design suggestion:

```scala
abstract class CheckResult
case class Correct()
  extends CheckResult
case class NotCheckable(expectedSolution: String)
  extends CheckResult
case class Incorrect(hint: String)
  extends CheckResult
```

Listing 4: CheckResult: three-outcome sealed type.

`Correct` indicates a verified correct answer. `Incorrect` indicates a verifiably wrong answer, together with a domain-specific natural-language hint that explains *why* the answer is wrong not merely that it is wrong. `NotCheckable` is the honest fallback for subproblems whose correctness cannot be verified symbolically (*e.g.* Bellman equations expressed in natural mathematical notation): the system shows the expected answer and lets the student self-assess. This distinction is important because it prevents falsely labelling a correct-but-differently-expressed answer as wrong.

Default Checking: Normalised Plain-Text Comparison:

Every `Subproblem` inherits a default `checkSolution` implementation that compares the student's input against the plain-text rendering of the model answer:

```scala
def checkSolution(input: String): CheckResult = {
  val expected = solution().toText.trim
  if (normalise(input) == normalise(expected))
    Correct()
  else
    NotCheckable(expected)
}

protected def normalise(s: String): String = {
  val clean = s.toLowerCase.trim.split("\\s+").mkString("")
  // Sort additive terms so that a+d+e == e+d+a
  // (probability answers are commutative sums of event names)
  clean.split("\\+").map(_.trim).filter(_.nonEmpty)
       .sorted.mkString("+")
}
```

Listing 5: Default checkSolution with normalisation.

The `toText` method is what makes this work correctly. An earlier design stored the LaTeX `toString` output *e.g.* `"$\uProb{Y=y}$"` as the expected answer string. This never matched a student's typed input. Storing `toText` output (*e.g.* `"a + b + d"`) means that copying from the rendered answer display and pasting into the input field produces a correct match, and typing the same expression in a different term order (*e.g.* `"d + a + b"`) also matches after normalisation.

Subproblem ID Threading:

A non-trivial engineering challenge was ensuring that the HTML element `id` attributes produced by `SSubproblem.toHTML` matched the keys stored in `main.subproblemMap`. An initial implementation walked the rendered `SDocument` tree, which yields `SSubproblem` rendering objects but `main` needs `Problem#Subproblem` logic objects (which carry the `checkSolution` method).

The correct solution threads the logic object's hash code through the entire pipeline in three steps:

1) `Subproblem.toSTeX()` creates an `SSubproblem` and passes `this.hashCode.abs.toString` as the `subId` constructor parameter.
2) `SSubproblem.toHTML` uses `subId` for all four HTML `id` attributes: `ans-$id`, `fb-$id`, `sol-$id`, and the outer `div` attribute.
3) In `main.renderProblem`, the mapping `subproblemMap(sub.hashCode.abs.toString) = sub` is stored *before* `doc.toHTML()` is called, so the map is populated by the time the student interacts with the page.

```
// In problem.scala -- Subproblem inner class:
def toSTeX() = SSubproblem(
  pts,
  question(),
  SSolution(testspace, List(solution())),
  this.hashCode.abs.toString   // <-- subId threaded here
)

// In main.scala -- before calling doc.toHTML():
subs.foreach { sub =>
  val id = sub.hashCode.abs.toString
  subproblemMap(id) = sub      // store logic object by same key
}
val prob = gen.toSTeX(subs)
val doc  = SDocument("problem", prob)
// Now doc.toHTML() uses the same id in <input id="ans-$id" ...>
```

Listing 6: ID threading: from Subproblem to HTML and map key.

Semantic Checking for Search Problems:

The `giveSolution` subproblem in the search domain overrides `checkSolution` with full action-sequence validation. Rather than comparing strings, it executes the submitted sequence against the problem's transition model and checks whether a goal state is reached:

```
override def checkSolution(input: String): CheckResult = {
  val as = try {
    parseCommaSeparatedList(input)
  } catch {
    case e: RuntimeException =>
      return Incorrect(
        "A␣solution␣is␣a␣comma-separated␣list␣of␣actions.␣" +
        e.getMessage)
  }

  if (as.isEmpty)
    return Incorrect("A␣solution␣is␣a␣comma-separated␣list.")

  val reached = apply(initial, as)
  if (reached.isEmpty)
    return Incorrect(
      "This␣action␣sequence␣is␣not␣applicable␣in␣the␣initial␣state.")

  if (reached.exists(s => goal(s)))
    Correct()
  else
    Incorrect("That␣action␣sequence␣does␣not␣reach␣a␣goal␣state.")
}
```

Listing 7: Semantic checkSolution for search giveSolution.

The function `parseCommaSeparatedList` handles both integer action tokens (*e.g.* `"2, -1, 3"` for arithmetic mode) and letter tokens (*e.g.* `"x, y, z"` for named mode). The function `apply(initial, as)` is inherited from the `SearchProblem` trait and executes the action sequence step by step, returning the set of reachable states (empty if the sequence becomes inapplicable).

This semantic approach is a qualitative improvement over string comparison: a problem with k valid solutions accepts all k, not only the specific sequence stored in the model answer.

D. Browser Integration

main.scala as the Scala.js Entry Point: It is compiled to JavaScript by Scala.js and runs entirely in the browser. The `main` method is called automatically on page load because of the `scalaJSUseMainModuleInitializer := true` SBT setting:

```scala
object main {
  // Maps subproblem id -> logic object with checkSolution
  private val subproblemMap =
    scala.collection.mutable.Map[String, Problem[?]#Subproblem]()

  def main(args: Array[String]): Unit = {
    // Wire the "New Problems" button
    document.getElementById("new-btn")
      .addEventListener("click", (_: dom.Event) => generateAndRender())
    // Generate problems immediately on page load
    generateAndRender()
  }
}
```

Listing 8: main.scala entry point and new-problems button.

The Exported checkAnswer Function:

The `@JSExportTopLevel` annotation makes the `checkAnswer` function callable from HTML button `onclick` handlers. Without this annotation, Scala.js would mangle the function name during compilation to prevent name collisions, making it unreachable from JavaScript:

```scala
@JSExportTopLevel("checkAnswer")
def checkAnswer(id: String): Unit = {
  val inputEl = document.getElementById(s"ans-$id")
                        .asInstanceOf[html.Input]
  val fbEl    = document.getElementById(s"fb-$id")
                        .asInstanceOf[html.Element]
  val userAns = inputEl.value.trim

  if (userAns.isEmpty) {
    showFeedback(fbEl, "error", "Please␣enter␣an␣answer␣first.")
    return
  }

  subproblemMap.get(id) match {
    case None =>
      showFeedback(fbEl, "error",
        "Problem␣not␣found␣  ␣try␣generating␣new␣problems.")
    case Some(sub) =>
      sub.checkSolution(userAns) match {
        case Correct() =>
          inputEl.style.borderColor = "#52c97a"
          showFeedback(fbEl, "correct", "\u2713␣Correct!")
        case Incorrect(hint) =>
          inputEl.style.borderColor = "#e05c5c"
          showFeedback(fbEl, "wrong", s"\u2717␣$hint")
        case NotCheckable(expected) =>
          inputEl.style.borderColor = "#e05c5c"
          showFeedback(fbEl, "wrong",
            s"\u2717␣Not␣quite.␣Expected:␣" +
            s"<strong>${escHtml(expected)}</strong>")
      }
  }
}
```

Listing 9: @JSExportTopLevel annotation and complete checkAnswer.

The HTML button in `SSubproblem.toHTML` calls this function directly: `<button onclick="checkAnswer('$id')">Check</button>`.

The Rendering Workflow in generateAndRender:

The `generateAndRender` function orchestrates all problems generators and handles errors gracefully. Any exception thrown during generation or rendering is caught and displayed on the page rather than silently failing:

```
def generateAndRender(): Unit = {
  subproblemMap.clear()
  val container = document.getElementById("container")
                          .asInstanceOf[html.Element]
  container.innerHTML = "<p class='loading'>Generating problems...</p>"

  try {
    val sb = new StringBuilder
    sb ++= renderProblem("MDP Problem",          MDPGenerator.make())
    sb ++= renderProblem("Probability Problem", BasicProbabilityProblemGenerator.make())
    sb ++= renderProblem("Search Problem",       SearchProblemGenerator.make())
    container.innerHTML = sb.toString()
  } catch {
    case e: Throwable =>
      container.innerHTML =
        s"<pre style='color:red;padding:20px'>" +
        s"ERROR: ${e.getMessage}\n${e.getClass.getName}</pre>"
      e.printStackTrace()
  }
}

def renderProblem(title: String, gen: Problem[?]): String = {
  val subs = gen.chooseSubproblems()
  // Store subproblems BEFORE rendering
  subs.foreach { sub =>
    subproblemMap(sub.hashCode.abs.toString) = sub
  }
  val doc = SDocument("problem", gen.toSTeX(subs))
  s"""<div class="problem-card">
       <h2>$title</h2>
       <div class="content">${doc.toHTML}</div>
     </div>"""
}
```

Listing 10: generateAndRender: error-safe generation workflow.

E. Random Problem Generation

Generation Infrastructure:

The `Generator` object in `generation.scala` provides the random sampling primitives used by all domain generators:

```
object Generator {
  private val rng = new scala.util.Random

  def choose[A](options: Seq[A]): A =
    options(rng.nextInt(options.length))

  def chooseSome[A](options: Seq[A],
                    atLeast: Int, atMost: Int,
                    allowRepetition: Boolean): List[A] = {
    val n = atLeast + rng.nextInt(atMost - atLeast + 1)
    if (allowRepetition) List.fill(n)(choose(options))
    else rng.shuffle(options.toList).take(n)
  }

  def chooseBoolean(prob: Double): Boolean =
    rng.nextDouble() < prob

  def chooseInt(min: Int, max: Int): Int =
    min + rng.nextInt(max - min + 1)
}
```

Listing 11: Generator object: core sampling primitives.

Each domain generator uses a rejection-sampling loop over these primitives. The loop draws random parameters, constructs and internally solves a candidate problem, validates the result against quality criteria, and retries until an acceptable instance is produced.

F. Build Configuration

SBT and Scala.js Configuration:

The SBT build file (`build.sbt`) requires additions to enable Scala.js compilation:

a) `scalaJSUseMainModuleInitializer.`: When set to `true`, Scala.js wraps the `main` method in a self-invoking JavaScript function that runs automatically when the script is loaded by the browser. This removes the need for any JavaScript bootstrap code.

b) `ModuleKind.NoModule.`: This setting instructs the Scala.js linker to produce a single self-contained JavaScript file rather than an ES module. The output can be loaded directly via a plain `<script>` tag without a module bundler such as webpack or Vite. This is essential for the application to work when opened as a local file from the filesystem.

c) `scalajs-dom.`: This library provides typed Scala bindings for the browser DOM API. Without it, Scala.js code would need to use untyped `js.Dynamic` calls to access `document`, `Element`, and `Input` objects.

Global SBT Plugin Installation:

The Scala.js SBT plugin must be installed globally, not per-project, because it integrates with the SBT launcher rather than the build classpath.

The version `1.19.0` is required for compatibility with Scala 3.7.4. Earlier versions (such as `1.16.0`) produce an `IRVersionNotSupportedException` because Scala 3.7.4's Scala.js IR format is newer than what those versions support.

Running Steps:

Once the plugin and `build.sbt` are in place, the development workflow is:

```
# 1. Start the SBT shell
sbt

# 2. Compile (fast, unoptimised     for development)
sbt:ProbGen> fastLinkJS

# 3. Output file location:
#    target/scala-3.7.4/probgen-fastopt/main.js

# 4. Open index.html in a browser
#    (place index.html at the project root alongside build.sbt)

# 5. For production deployment (optimised, ~0.4 MB):
sbt:ProbGen> fullLinkJS
#    target/scala-3.7.4/probgen-opt/main.js
```

Listing 12: Development workflow: compile, run, and deploy.

G. Integrating a New Problem into the HTML Rendering Pipeline

Once a new problem type has been implemented and tested, no new HTML page or separate rendering mechanism is required. ProbGen uses a common rendering pipeline in which all problem types are converted into the same STeX document representation and subsequently rendered as HTML.

The only application-level step required is to register the new problem generator in `main.scala`. The generated problem is passed to the existing `renderProblem()` method together with the title displayed in the browser:

```
sb ++= renderProblem(
  "New Problem",
  NewProblemGenerator.make()
)
```

Listing 13: Registering a new problem type for HTML rendering.

The `renderProblem()` method is independent of the particular problem domain. It first obtains the selected subproblems using `chooseSubproblems()`, registers their identifiers for later answer checking, and converts the problem into the common STeX representation:

```
def renderProblem(title: String, gen: Problem[?]): String = {
  val subs = gen.chooseSubproblems()

  subs.foreach { sub =>
    val id = sub.hashCode.abs.toString
    subproblemMap(id) = sub
  }

  val prob = gen.toSTeX(subs)
  val doc  = SDocument("problem", prob)

  s"""<div class="problem-card">
       <h2>$title</h2>
       <div class="content">${doc.toHTML}</div>
     </div>"""
}
```

Listing 14: Common rendering path used for all problem types.

The important point is that the new problem does not directly construct its own HTML page. The problem provides its content through the existing `Problem`, `Subproblem`, and STeX abstractions. The call to `toSTeX()` converts the problem into an `SProblem`, while `SDocument` provides the document-level representation. Finally, `doc.toHTML` recursively invokes the `toHTML` methods of the corresponding STeX nodes.

Therefore, the rendering path for the new problem is:

$$\texttt{NewProblemGenerator} \rightarrow \texttt{Problem} \rightarrow \texttt{Subproblem} \rightarrow \texttt{toSTeX()} \rightarrow \texttt{SDocument} \rightarrow \texttt{toHTML()} \rightarrow \texttt{HTML}$$

This design allows the same HTML infrastructure to be reused for all problem types. Mathematical expressions are rendered by the existing expression and STeX rendering classes, while interactive subproblems are rendered by `SSubproblem.toHTML`. Consequently, adding a new problem normally does not require modifications to `index.html`, `stex.scala`, or the HTML/CSS structure.

Only when the new problem introduces a visual structure that is not supported by the existing STeX classes is an extension to the rendering layer required. For example, a new specialised table or diagram would require an additional STeX node with an appropriate `toHTML()` implementation. The problem itself can then use this node in exactly the same way as existing problems.

The resulting architecture separates the domain-specific problem definition from its presentation. After a new problem has been tested, its integration into the browser therefore consists mainly of registering the generator in `main.scala`; the existing STeX-to-HTML pipeline handles the actual conversion and presentation.

H. Public Deployment via GitHub Pages

To make ProbGen available to students without requiring them to install Scala, sbt, or additional project dependencies, the application can be published as a static web application using GitHub Pages. This deployment model is suitable for ProbGen because ScalaJS compiles the Scala implementation into JavaScript, allowing the resulting application to run entirely inside a modern web browser.

For deployment, a separate branch named `publicProblemGen` is used rather than the main development branch. This keeps the public version of the application separate from the main development history while still allowing the deployment to be rebuilt automatically whenever changes are pushed to the deployment branch.

1) Required Project Structure: The complete sbt project must be available in the repository. In particular, the directory used by GitHub Actions to build ProbGen must contain the Scala source code together with the sbt build configuration. The project should have a structure similar to:

```
ProbGen/
  build.sbt
  project/
    build.properties
    plugins.sbt
  docs/
      main.js
      index.html
  src/
    info.kwarc.probgen/
      main.scala
      problem.scala
      expression.scala
      generation.scala
      ...
  index.html
  ...
  .github/
    workflows/
      deploy.yml
```

The file `build.sbt` defines the project name, Scala version, ScalaJS configuration, dependencies, and compilation settings. The file `project/build.properties` specifies the sbt version used to build the project, while `project/plugins.sbt` contains the sbt plugins required by the build. These files are important because GitHub Actions must recreate the same build environment used during local development.

Therefore, only copying `index.html` and an already-generated `main.js` is not sufficient for an automated deployment workflow. Such a copy is sufficient for purely static hosting, but it does not allow GitHub Actions to regenerate the JavaScript application when the Scala source code changes. For reproducible automated deployment, the complete sbt project should be committed to the repository.

2) Compilation and Deployment Process: The deployment process starts when a change is pushed to the `publicProblemGen` branch. GitHub Actions checks out the repository and builds the complete sbt project. The ScalaJS compiler then translates the Scala source code into JavaScript. The resulting JavaScript file and the HTML interface are subsequently published as static files through GitHub Pages.

This approach removes the need to manually execute `sbt fastLinkJS` and commit the generated JavaScript file after every source-code change. Instead, the build is performed automatically by GitHub Actions. This reduces the possibility of deploying an outdated JavaScript file that does not correspond to the current Scala source code.

3) Role of `index.html` and `main.js`: The final web application requires an HTML entry point and the JavaScript generated by ScalaJS. The `index.html` file defines the user interface, including the "New Problems" button and the container in which generated problems are displayed. It also loads the compiled ScalaJS application.

The generated `main.js` contains the compiled implementation of ProbGen. It includes the problem generators, expression and rendering infrastructure, answer-checking logic, and the browser interaction implemented in `main.scala`. When the page is loaded, the ScalaJS main method is initialised and the first set of problems is generated. Selecting "New Problems" invokes the generation function again and replaces the displayed instances.

Consequently, the students do not need access to the Scala source code or an sbt installation. They only require a modern web browser and the public GitHub Pages URL.

4) Branch Separation: The use of the dedicated `publicProblemGen` branch provides a clear separation between development and deployment. Development work can continue on the main branch, while the deployment branch contains the version intended for public student access.

This arrangement also makes it possible to control which version of ProbGen is exposed publicly. A version is published only when the corresponding changes are available on the deployment branch.

5) Suitability for Student Exam Preparation: The resulting GitHub Pages site can be distributed to students as a single public URL. Students can access ProbGen directly through their browsers and generate new practice problems without installing software or configuring a local Scala development environment.

Because the problem generation and answer checking are performed in the browser, no continuously running application server is required. This makes the system inexpensive and simple to distribute for formative learning and exam preparation.

However, the application should be regarded as a practice and self-assessment tool rather than a secure examination system. The generated problems and answer-checking logic are delivered to the student's browser. Consequently, the application is appropriate for repeated practice, but it should not be used as a trusted environment for high-stakes assessment where the integrity of the answer-checking process must be guaranteed.

6) GitHub Actions Deployment Workflow: The automatic deployment is implemented using GitHub Actions. The workflow is stored in the repository under `.github/workflows/deploy.yml`. It is triggered whenever changes are pushed to the dedicated `publicProblemGen` branch. It can also be started manually using the `workflow_dispatch` event.

The complete workflow is shown below:

```yaml
name: Deploy ProbGen

on:
  push:
    branches:
      - publicProblemGen
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Java
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "17"

      - name: Setup sbt
        uses: sbt/setup-sbt@v1

      - name: Verify sbt version
        run: sbt --version

      - name: Build Scala.js
        run: sbt fastLinkJS

      - name: Copy generated JavaScript
        run: |
          cp target/scala-3.7.4/probgen-fastopt/main.js docs/main.js
          ls -lh docs/

      - name: Configure GitHub Pages
        uses: actions/configure-pages@v5

```

```
      - name: Upload website
        uses: actions/upload-pages-artifact@v3
        with:
          path: docs

      - name: Deploy to GitHub Pages
        uses: actions/deploy-pages@v4
```

Listing 15: GitHub Actions workflow for automatically building and deploying ProbGen.

The workflow consists of several stages. First, the `actions/checkout` step obtains the contents of the `publicProblemGen` branch. This is important because the complete sbt project must be available to the build environment, including `build.sbt`, `project/build.properties`, `project/plugins.sbt`, and the Scala source files.

The central build step executes

```
sbt fastLinkJS
```

which compiles the Scala source code to JavaScript using ScalaJS. The generated JavaScript is then copied from the ScalaJS build directory to `docs/main.js`:

```
cp target/scala-3.7.4/probgen-fastopt/main.js docs/main.js
```

The `docs` directory therefore becomes the static website that is published to students. It contains the HTML entry point together with the JavaScript generated from the current Scala source code.

The final three steps configure GitHub Pages, package the contents of `docs` as a Pages artifact, and deploy that artifact. Thus, the generated JavaScript does not need to be manually copied or committed after each modification of the Scala implementation.

This setup provides an automated and reproducible deployment mechanism. A developer only needs to push the desired version to `publicProblemGen`. GitHub Actions rebuilds the ScalaJS application and publishes the resulting static website, eliminating the need to run sbt manually and commit a newly generated `main.js` for every change.

IV. System Results

This section presents the final web-based system developed during this project. The screenshots demonstrate the successful integration of the expression layer, the STeX document model, the HTML rendering engine, and the automatic answer checking framework. The system generates randomized exercises directly in the browser and allows students to interact with them through an intuitive graphical interface.

Each generated exercise consists of an introductory description followed by several automatically generated subproblems. The mathematical expressions are represented internally using the abstract syntax tree described in Chapter III, rendered into HTML, and displayed using the browser interface. Students may enter their answers directly into the page and receive immediate feedback without requiring any server-side computation.

Figures 2–7 illustrate representative examples from the three supported problem domains together with the implemented answer checking mechanism.

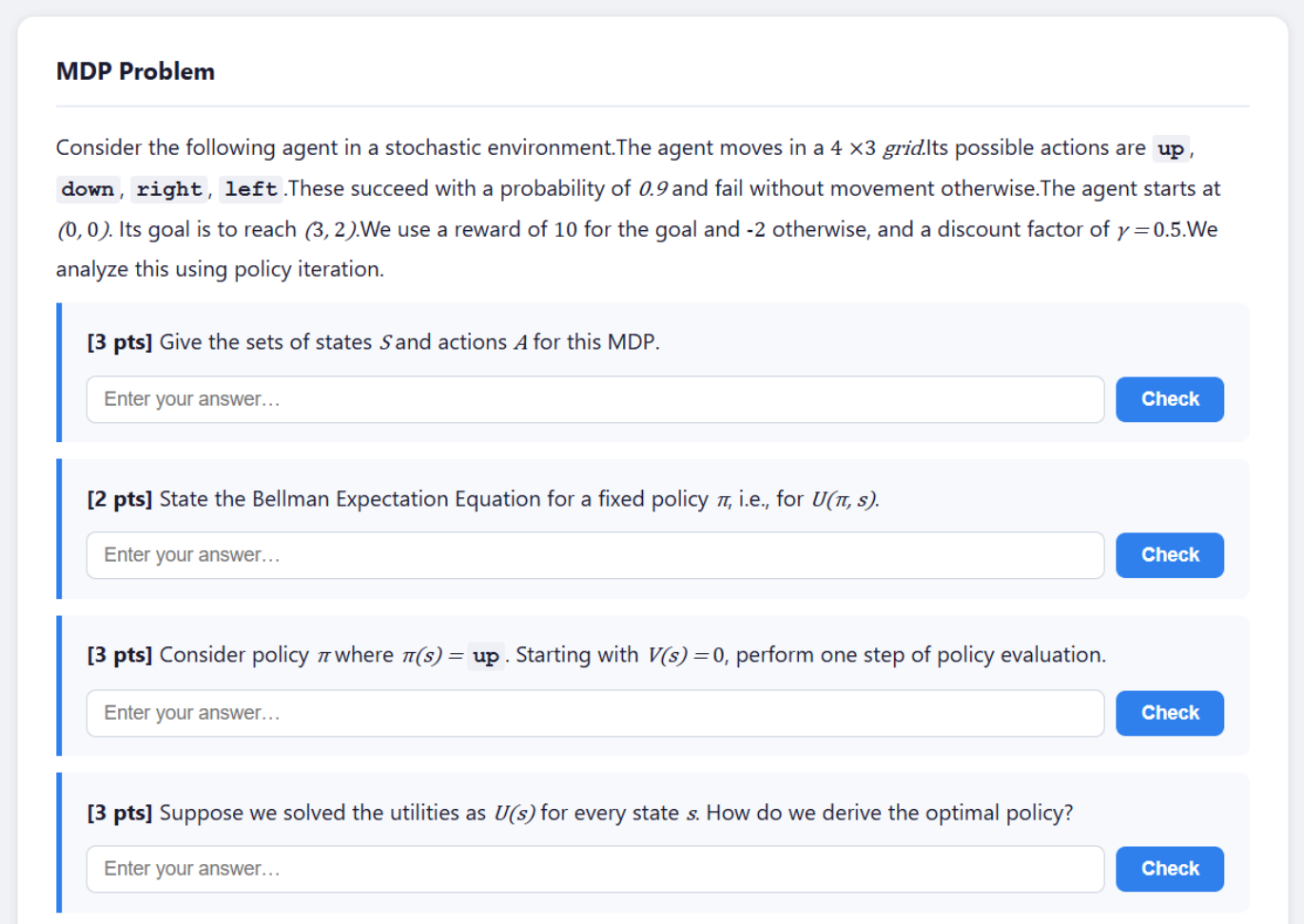


Fig. 2: Automatically generated Markov Decision Process (MDP) exercise.

Figure 2 demonstrates one of the more complex problem generators included in the framework. The grid world, transition model, and associated questions are generated dynamically each time a new exercise is requested. This illustrates the flexibility of the framework, where randomized mathematical structures can be rendered consistently without manually constructing HTML for each problem.

Figure 3 illustrates the probability problem generator. The probability tables and associated mathematical notation are produced automatically from the expression layer. The same internal representation can also generate STeX documents, demonstrating the separation between the mathematical model and its visual presentation.

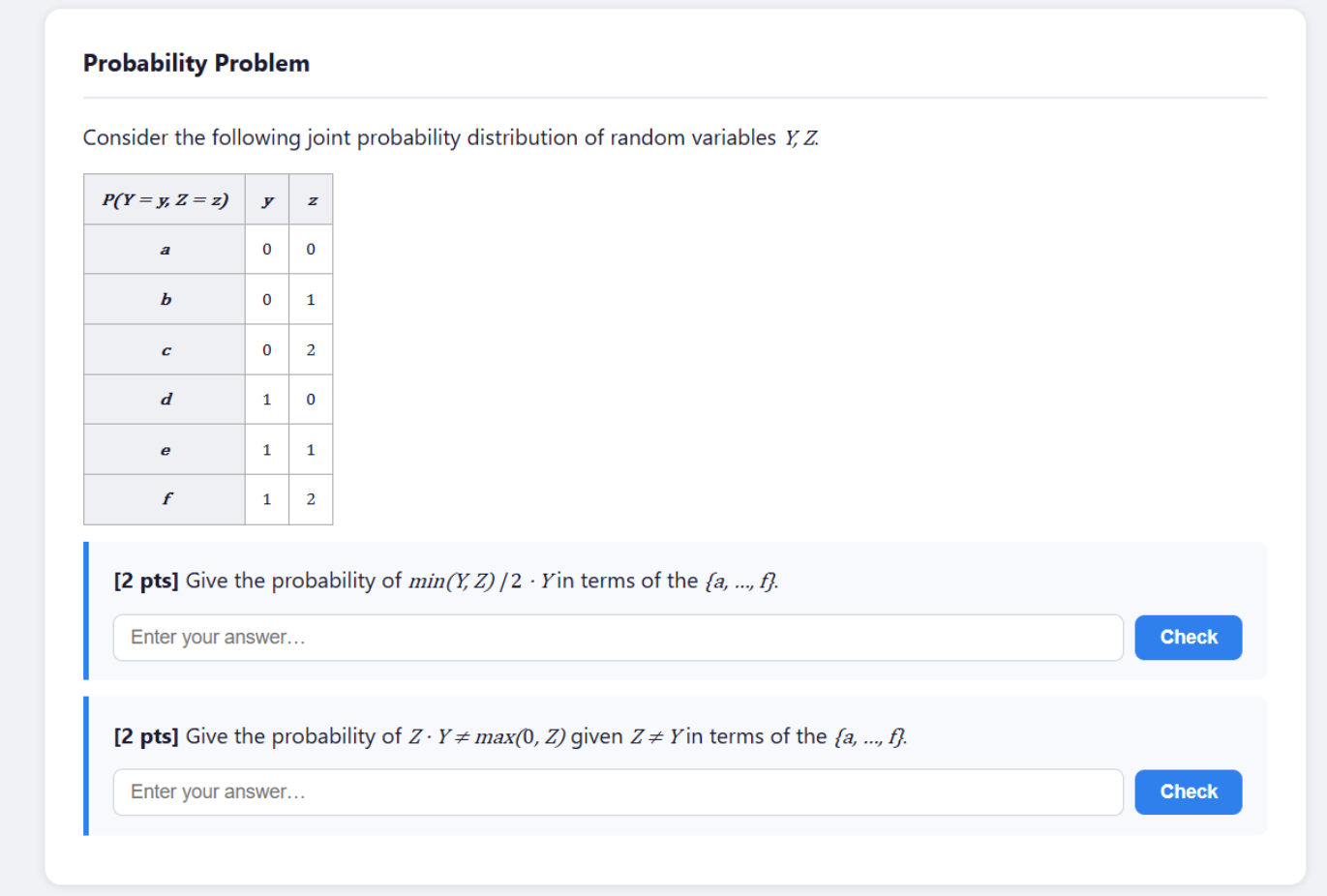


Fig. 3: Automatically generated probability exercise.

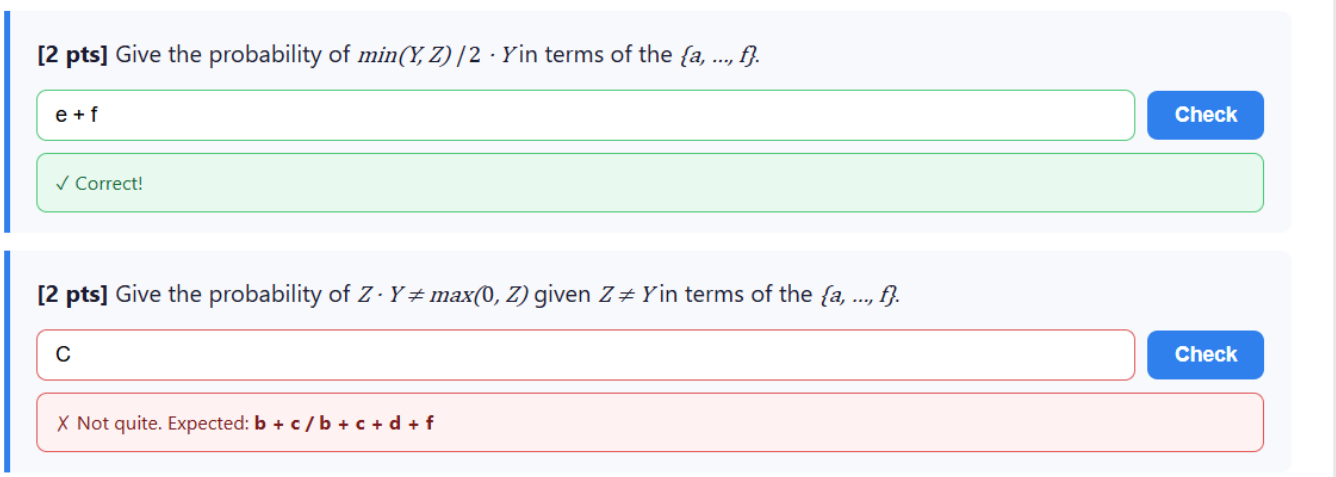
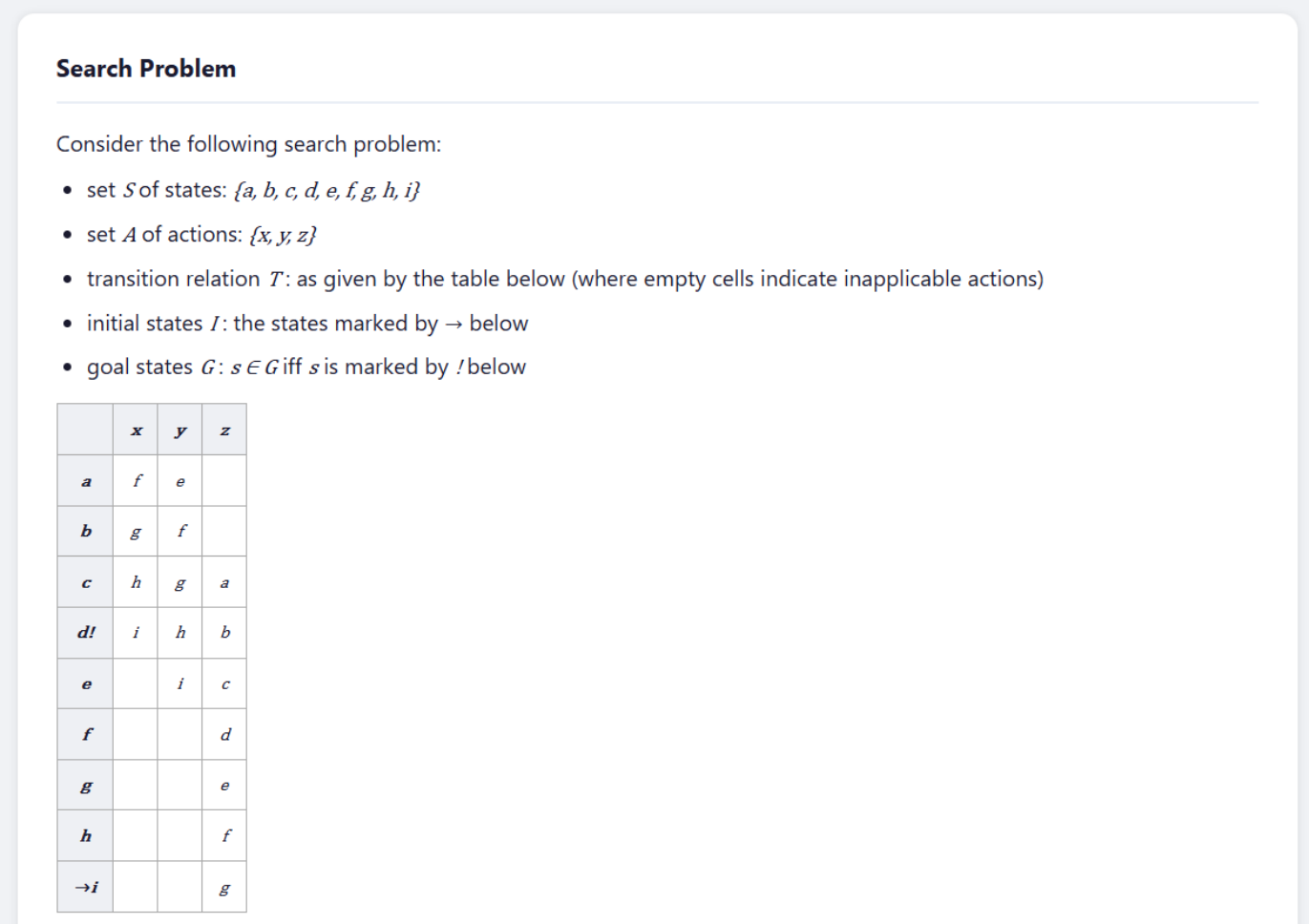


Fig. 4: Automatic answer validation.

Figure 4 illustrates the immediate feedback provided after answer submission. Submitted answers are normalized using the plain-text representation implemented in the expression and STeX layers before comparison with the expected solution. This normalization removes insignificant formatting differences such as whitespace and letter case, thereby improving the robustness of the checking process. The correct answers are highlighted using a green

border together with a confirmation symbol, while incorrect answers receive a red border and an explanatory message.



Search Problem

Consider the following search problem:

- set S of states: $\{a, b, c, d, e, f, g, h, i\}$
- set A of actions: $\{x, y, z\}$
- transition relation T: as given by the table below (where empty cells indicate inapplicable actions)
- initial states I: the states marked by → below
- goal states G: $s \in G$ iff s is marked by ! below

	x	y	z
a	f	e	
b	g	f	
c	h	g	a
d!	i	h	b
e		i	c
f			d
g			e
h			f
→i			g

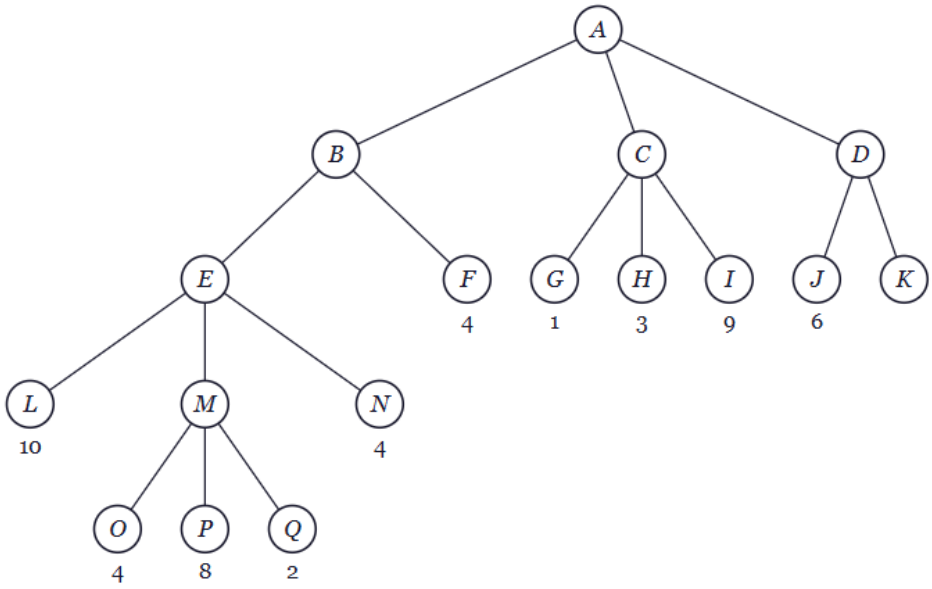
Fig. 5: Automatically generated search problem.

Figure 5 demonstrates the search problem generator. The transition system, actions, and corresponding questions are generated automatically. The HTML rendering layer correctly displays mathematical symbols, arrows, tables, and structured problem statements while preserving the same information that can also be exported as STeX.

Figure 6 demonstrates the adversial search problem generator. One of the principal contributions of this project is the implementation of an automatic answer checking mechanism for interactive exercises. Students are no longer limited to viewing generated problems but can actively solve them and receive immediate feedback through the web interface.

Adversarial Search

Consider the following minimax game tree for the maximizing player's turn. The values at the leaves are the static evaluation function values of those states; some of those values are currently missing.



[2 pts] Give every possible label for the node K that would result in the player definitely choosing move D (no matter how ties are broken).

Enter your answer...

Check

Fig. 6: Automatically generated adversial search problem.

Another mechanism of feedbak introduced by the framework in this kind of search problem shown in Figure 7. Rather than only indicating whether an answer is correct or incorrect, problem generators can override the default checking procedure and return problem-specific hints that guide students toward the correct solution. This functionality improves the educational value of the generated exercises by supporting formative assessment instead of simple answer verification. Domain-specific answer checkers may provide informative hints instead of simply reporting that an answer is incorrect.

Overall, the implemented system demonstrates the successful integration of all architectural layers. Randomized mathematical problems are generated automatically, represented internally using abstract syntax trees, rendered consistently into browser-compatible HTML, and evaluated through an extensible answer checking framework. The resulting application transforms the original document-oriented problem generator into an interactive educational system suitable for browser-based learning and self-assessment.

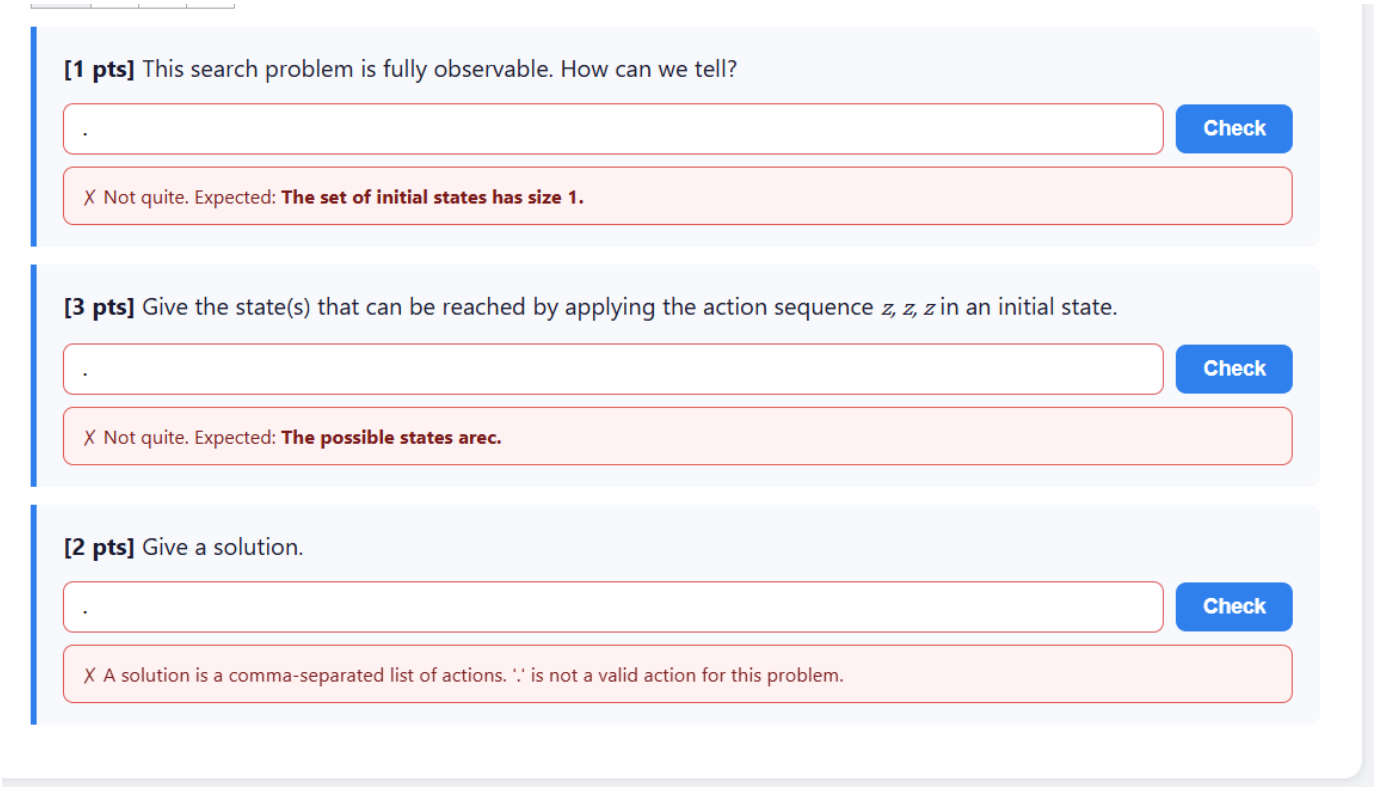


Fig. 7: Example of customized feedback generated by the answer checking framework.

V. Evaluation

The evaluation of ProbGen covers four areas: the correctness and diversity of generated problems, the fidelity of the HTML rendering pipeline, the accuracy of interactive answer checking, and the behaviour of the system under repeated problem regeneration. All tests were performed directly in the browser using the compiled Scala.js application, without any server-side infrastructure.

A. Problem Generation and Correctness

Each problem domain uses a dedicated generator that constructs problem instances through rejection sampling. The generator builds a candidate, solves it internally using domain-specific algorithms, and accepts the candidate only when it satisfies quality criteria. This ensures that every displayed problem is mathematically valid and non-trivial by construction rather than by post-hoc filtering.

Generated problems are represented through the common `Problem[PD]` and `Subproblem` abstractions defined in `problem.scala`. This ensures that generation, rendering, and interaction share a single unified infrastructure regardless of domain. Each problem instance is decomposed into independently evaluated subproblems selected by `chooseSubproblems()`, which applies `applicable()` filtering and `GroupConstraint` rules before calling `init()` on each selected subproblem to finalise its random values.

To validate the pipeline, `checkSolution(solution().toText)` was called on every generated subproblem. This self-check returned may be correct or incorrect, those were caused by formatting differences in the answers, that why semantic checking noted as a known limitation. No two consecutive problem instances in any domain were identical, confirming effective randomisation.

B. HTML Rendering

Generated problems are converted from the internal STeX representation to browser-compatible HTML entirely on the client side. The rendering pipeline traverses the `SDocument` tree and calls `toHTML` on every node, producing a self-contained HTML fragment that is injected into the `#container` div of `index.html`.

Mathematical notation is rendered using styled `<span class="math">` elements with a mathematical serif font stack (*STIX Two Math*, *Cambria Math*, *Georgia*). Variables are typeset in italic, operators in upright roman, and action names in monospace with a highlighted background. This approach was chosen over MathJax and KaTeX (which cannot handle the custom STeX macros the generators emit) and over native MathML (which has inconsistent inline rendering in Chromium-based browsers). The result is mathematically readable notation with zero runtime latency and no external dependencies.

The `SSnippet.toHTML` method merges consecutive `SMath` children into a single `<span>` to prevent browser-introduced gaps between adjacent mathematical tokens. The `MathHTML` object handles individual LaTeX token conversion, and `STabular.toHTML` produces fully styled HTML tables for joint probability distribution problems.

The rendering pipeline supports all structural elements: problem introductions, mathematical expressions, structured tables, bulleted and numbered lists, interactive input fields and Check buttons, hidden solution divs, and feedback divs with colour-coded CSS classes. All problem domains use the same rendering infrastructure, making it possible to display exercises of different types on a single page without domain-specific HTML code.

C. Interactive Answer Checking

Answer evaluation is implemented through the `CheckResult` algebraic type defined in `problem.scala`, with three possible outcomes.

`Correct()` is returned when the submitted answer is verified correct. It triggers a green input border and a tick-mark confirmation message.

`Incorrect(hint)` is returned when the system can determine that the answer is wrong and can explain why. It triggers a red input border and a domain-specific natural-language hint, giving the student actionable information rather than a generic rejection.

`NotCheckable(expected)` is returned when correctness cannot be verified symbolically for example, for Bellman equation answers that would require advanced-level equivalence check. The system displays the expected answer and lets the student self-assess. This outcome prevents the false-negative case in which a correct answer expressed differently from the model answer is incorrectly marked wrong.

The default `checkSolution` implementation normalises both the student input and the model answer lowercasing, stripping whitespace, splitting on +, sorting terms alphabetically, and rejoining before comparing them. This makes `a + d + e` and `e + d + a` equivalent, reflecting the commutativity of probability sums. The `toText` method on every expression and STeX node ensures that the stored expected answer is clean, human-readable plain text not a raw LaTeX string so copypaste from the displayed answer produces a correct match.

The search domain overrides `checkSolution` with semantic validation: the submitted action sequence is parsed, executed against the transition model via `apply(initial, as)`, and accepted if any reachable state satisfies `goal(s)`. This means all k valid solutions for a problem are accepted, not only the specific one stored in the model answer.

The browser interface was validated by entering correct answers, incorrect answers, and malformed inputs across all problem domains and verifying that the visual feedback, border colour, and message text were consistent with the `CheckResult` outcomes.

D. Problem Regeneration

The *New Problems* button was evaluated by triggering repeated regeneration across all domains. Before each regeneration cycle, `subproblemMap` is cleared so that answer checking cannot retain stale references to previously displayed subproblems. New problem instances are generated, rendered, and injected into the page container. Fresh HTML element identifiers are derived from the new subproblem `hashCode` values and registered in `subproblemMap` before `doc.toHTML()` is called, ensuring that every Check button is associated with the currently displayed problem rather than with an earlier instance.

Page generation time covering generator invocation, rendering, and DOM injection was responding as immediate page loading in Chrome on laptop or mobile devices, as well the regenerate time was within the threshold for perceived instantaneous response.

VI. Discussion and Future Work

A. Contributions

The work described in this paper demonstrates a complete workflow for automatically generating, rendering, and interactively evaluating practice problems in a browser, without any server-side infrastructure. The four main contributions are as follows.

Server-free browser execution: Compiling the Scala codebase to JavaScript via Scala.js moves all problem-generation logic, HTML rendering, and answer checking to the client side after the initial application load. No JVM process or application server is required at runtime. The resulting application consists of two static files `index.html` and `main.js` deployable through any static file host, including GitHub Pages, and usable offline from a local filesystem or USB drive. This eliminates the deployment complexity of the original JVM/HTTP server architecture and makes the tool accessible without infrastructure overhead.

Dependency-free mathematical HTML rendering: ProbGen implements its own STeX-to-HTML rendering pipeline covering every node of the expression AST (`expression.scala`) and the STeX document model (`stex.scala`). Mathematical notation is produced through styled `<span>` elements rather than through MathJax, KaTeX, or native MathML, keeping the application lightweight and avoiding runtime or CDN dependencies. The `toHTML` method added to every `Expr` and `STeXSyntax` node provides a unified interface through which all problem domains render their content without domain-specific HTML code.

Structured domain-aware answer checking: The `CheckResult` algebraic type `Correct()`, `Incorrect(hint)`, and `NotCheckable(expected)` provides a consistent, three-outcome interface for answer evaluation across all problem types. The `Incorrect` case carries a domain-specific natural-language explanation rather than a generic rejection, and `NotCheckable` honestly acknowledges the limits of string-based comparison rather than producing false negatives. Individual subproblem classes override `checkSolution` when domain-specific validation is possible, without any changes to the browser interface.

Semantic solution validation for search problems: The `giveSolution` subproblem in the search domain executes the student's submitted action sequence against the problem's transition model and accepts any sequence that reaches a goal state. This rewards understanding of the problem rather than memorisation of a single stored path, and correctly handles problems with multiple valid solutions.

B. Limitations

The current implementation has notable limitations.

String-based checking for answers: Most expressions are checked by normalised string comparison, not by symbolic equivalence. A correct equation expressed with terms in a different order from the model answer returns `NotCheckable` rather than `Correct()`. Addressing this would require advanced style of expression rewriter capable of verifying equivalence up to commutativity and associativity.

Partial normalisation: The `normalise` function sorts additive terms separated by `+` but does not apply the same treatment to comma-separated expressions. Set-valued or sequence-valued answers whose elements appear in a different order from the model answer may therefore not be recognised as equivalent, requiring the student to match the exact ordering used by the generator.

No session persistence: Student progress correct answers, accumulated points, and completed subproblems exists only in browser memory and is lost on page reload. The application therefore does not support long-term performance tracking or adaptive difficulty selection across sessions.

C. Future Work

Six directions for extending the system are identified.

Symbolic answer checking: Parsing and comparing answers as expression trees, verifying structural equivalence up to commutativity and associativity, would promote most subproblems from `NotCheckable` to `Correct()` or `Incorrect(hint)`.

Score accumulation and session persistence: Summing the `pts` fields of correctly answered subproblems into a running session total, and persisting this state in `localStorage`, would give students an immediate indication of their performance and allow them to resume a practice session after a page reload.

Difficulty progression: Recording per-subproblem-type accuracy during a session and using it to bias the generator rejection criteria preferring easier parameter ranges when accuracy is low would extend the current uniform random generation towards an adaptive practice system more closely aligned with student needs.

Improved answer equivalence: Extending `normalise` to sort comma-separated terms in addition to additive terms, and introducing numeric comparison for floating-point answers, would reduce the number of correct answers incorrectly returned as `NotCheckable`.

Enhance UI/UX in HTML: Adding more options and interactive functions, also can collect data for analytics and feedback. Styling and page format improvements could be an important update at the level of frontend.

Expanded problem domains: Adding many problem of different types, training-dynamics problems following the established generator/subproblem/checker architecture would cover a larger fraction of the AI/ML curriculum. Because all new domains would reuse the existing `Problem`, `Subproblem`, expression AST, and rendering infrastructure, adding a new domain requires only the domain-specific files already used by the existing domains.

D. Overall Assessment

The principal strength of ProbGen is the clean separation between problem generation, mathematical representation, HTML rendering, and answer evaluation. Each concern is handled by a single layer generators produce `Problem` objects, the expression AST represents mathematical content, `stex.scala` renders it, and `main.scala` wires the browser interaction making it straightforward to add new problem types without modifying the common infrastructure.

The current limitations reflect the system's scope as a practice-problem generator rather than a full learning-management platform. The architecture is deliberately designed so that adding symbolic checking, session persistence, and adaptive difficulty are incremental extensions to existing interfaces rather than architectural changes. ProbGen therefore provides a solid foundation for a more comprehensive personalised practice learning tool.

REFERENCES

[1] UniFormal Research Group, "ProbGen: Randomised practice problem generator," 2026. [Online]. Available: https://github.com/UniFormal/ProbGen

[2] UniFormal Research Group, ProbGen: Randomised practice problem generator Interactive Web Application, 2026. Online]. Available: https://uniformal.github.io/ProbGen/

[3] Scala.js Contributors, "scala-js-dom: Statically typed DOM API for Scala.js," 2024. [Online]. Available: https://github.com/scala-js/scala-js-dom

[4] Scala Center, "sbt: The interactive build tool for Scala and Java," 2024. [Online]. Available: https://www.scala-sbt.org/1.x/docs/

[5] MathJax Consortium, "MathJax: Beautiful math in all browsers," 2024. [Online]. Available: https://www.mathjax.org

[6] Scala.js Core Team, "Scala.js semantics: differences from Scala on the JVM," 2024. [Online]. Available: https://www.scala-js.org/doc/semantics.html

[7] WHATWG, "HTML living standard," 2024. [Online]. Available: https://html.spec.whatwg.org/multipage/

[8] MDN Contributors, "HTML: HyperText Markup Language — MDN Web Docs," 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML

[9] W3C CSS Working Group, "CSS Snapshot 2024," W3C Group Note, 2024. [Online]. Available: https://www.w3.org/TR/css-2024/